C for Pebble Programmers - Emery course syllabus

Version: revised 41-activity self-directed syllabus - Target: Pebble Time 2 (Emery, 200 x 228) - Language: C - SDK: 4.33.1 - Focused study estimate: 52h 35m - Schedule: self-paced with prerequisite gates.

Course purpose

Learn to design, implement, understand, test and independently extend Pebble apps. This is not merely a catalog of SDK functions: the technical capabilities support seven essential competencies, revisited throughout the course. You progress from a time-only watchface to three evolving watchapps and finally to an independently conceived and built app. No university enrollment, attendance, grade, or fixed deadline is implied.

Seven learning outcomes

By the end of the course, demonstrate all seven abilities independently:

UI Design
Measurable outcome: Create legible Emery layouts, navigate between screens, and communicate states and actions clearly.

Idea Generation
Measurable outcome: Develop alternatives, evaluate constraints, and justify an original app concept or representation.

Algorithmic Thinking
Measurable outcome: Trace execution, specify state/loop logic, and validate state transitions and boundary conditions.

C Programming
Measurable outcome: Write and explain C data/control flow, pointers, callback functions, object ownership, and resource safety.

Resource Literacy
Measurable outcome: Interpret SDK signatures and limits, locate sources, and make evidence-based API choices.

Debugging and Testing
Measurable outcome: Reproduce issues, form hypotheses, make minimal fixes, and verify regression and negative tests.

Development Tools and Workflow
Measurable outcome: Build, test, log, use Git, manage integration artifacts, and reproduce results independently.

Each of the 40 preparatory activities has a primary and supporting competency; the capstone is evaluated against all seven. Primary task distribution: 10 C programming; 5 each in the other six competencies. Hours may vary by task; equal task counts do not imply equal time.

Complete outcomes, evidence and capstone rubric - All 41 activities mapped to competencies

What you need

Pebble SDK 4.33.1, the Emery emulator, C build tools, Git, editor, and local source files. Use an actual Pebble Time 2 where sensor calibration, vibration, backlight or physical device behavior must be confirmed; phone communications also need a connected compatible Pebble mobile environment. Do not present an emulator result as physical hardware verification. Official C reference.

Start here: Lesson index - C readings 3A-24H - Pebble API track - Taskwarrior instructions.

Learning method and assessment

For every lesson: (1) Read and plan: study the linked local reading/SDK signatures and express the intended change in your own words. (2) Implement and test: work from the local starter, commit your change, build/run, and test a normal plus a boundary/failure case. (3) Reflect and transfer: explain state and resource ownership, record a problem or trade-off, and name one additional use. Existing activities already specify their deliverables; these steps organize rather than replace them.

Score the activity's primary and supporting competency at 0-3 using the common rubric. Both must reach 2 or 3, and the stated acceptance criteria must pass, before marking the activity complete. Do not conflate unverified hardware behavior with an implemented fallback.

0 - Not demonstrated
Standard: No functional evidence or intelligible explanation

1 - Developing
Standard: Partially demonstrated, with critical understanding/test gaps

2 - Competent
Standard: Correct or appropriately scoped result, independent explanation, normal and boundary tests, and saved evidence

3 - Transferable
Standard: Competent plus independent generalization to a new context with justified trade-offs

Independent-work policy: Do not use AI-generated source, solutions, code explanations, diagrams or debugging fixes to complete exercises. Consult included starters, official SDK documentation, logs, compiler diagnostics and your own notes. Versioned evidence is required. Detailed scoring and readiness gates.

Optional self-check (pilot): After you finish selected activities (2A, 3A in v1), you may run the local assessor tool (assess submit <ID>) to compare your Git-backed evidence and EVIDENCE.md against course criteria and meets/below exemplars. It uses spaCy, not generative AI, does not help during exercises, and does not mark Taskwarrior tasks complete.

Recommended integrated course progression

The sequence interleaves C learning with UI work, ideation, algorithm design, research, debugging and tools. The API material no longer sits as a detached final block. Preserve all original IDs, and use the "Depends on" column and Taskwarrior to work out of order safely.

I Orientation and fundamentals
7 | 4h 45m

II Control flow, code reading and first window
9 | 10h 45m

III Memory, design, events and interactive foundations
14 | 19h 00m

IV Graphics, hardware, validation and integration
10 | 14h 05m

V Independent application capstone
1 | 4h 00m

Phase I - Orientation and fundamentals

Readiness checkpoint: Working time-only watchface, baseline Git workflow and independent diagnostic explanation.

1A Explore Pebble examples
Duration: 20m | Primary: Idea Generation | Supporting: Resource Literacy
Depends on: None

2A Build and run a time-only C watchface
Duration: 30m | Primary: Development Tools and Workflow | Supporting: C Programming
Depends on: None

3A Variables, Data Types, and Operators
Duration: 120m | Primary: C Programming | Supporting: Debugging and Testing
Depends on: 2A

4A Find and document a Pebble API function
Duration: 20m | Primary: Resource Literacy | Supporting: C Programming
Depends on: 3A

5A Change one visual property of a time-only watchface
Duration: 30m | Primary: UI Design | Supporting: Resource Literacy
Depends on: 3A, 4A, 2A

6A Diagnose a simple compiler error
Duration: 20m | Primary: Debugging and Testing | Supporting: Development Tools and Workflow
Depends on: 3A, 2A

7B Create a Git-based modification workflow
Duration: 45m | Primary: Development Tools and Workflow | Supporting: Debugging and Testing
Depends on: 2A, 5A, 6A

Phase II - Control flow, code reading and first window

Readiness checkpoint: Explain control flow, trace callbacks, design an initial UI and articulate a bounded algorithm.

8B Conditional Execution and Decision Making
Duration: 120m | Primary: C Programming | Supporting: Algorithmic Thinking
Depends on: 3A

9C Loops and Iteration
Duration: 135m | Primary: C Programming | Supporting: Algorithmic Thinking
Depends on: 8B

10D Functions and Program Structure
Duration: 150m | Primary: C Programming | Supporting: Algorithmic Thinking
Depends on: 9C

11A Trace the watchface lifecycle
Duration: 30m | Primary: Algorithmic Thinking | Supporting: C Programming
Depends on: 10D, 2A

12A Windows and layers
Duration: 60m | Primary: UI Design | Supporting: Development Tools and Workflow
Depends on: 2A, 11A

13B Add and style a date TextLayer
Duration: 45m | Primary: UI Design | Supporting: C Programming
Depends on: 5A, 10D, 4A

14B Trace an existing feature through source code
Duration: 45m | Primary: Resource Literacy | Supporting: Algorithmic Thinking
Depends on: 11A, 10D, 4A

15B Design a font-cycling algorithm
Duration: 30m | Primary: Algorithmic Thinking | Supporting: UI Design
Depends on: 11A, 14B

16B Write a two-example remix brief
Duration: 30m | Primary: Idea Generation | Supporting: UI Design
Depends on: 1A, 14B

Phase III - Memory, design, events and interactive foundations

Readiness checkpoint: Demonstrate C memory/state, original concept and interactive app behavior with tests.

17E Arrays, Strings, and Bounds
Duration: 90m | Primary: C Programming | Supporting: Debugging and Testing
Depends on: 10D

18F Pointers, Indirection, and Memory Lifetime
Duration: 105m | Primary: C Programming | Supporting: Debugging and Testing
Depends on: 17E

19G Structures, Enums, and Program State
Duration: 105m | Primary: C Programming | Supporting: Algorithmic Thinking
Depends on: 18F

20C Research an unfamiliar Pebble capability
Duration: 60m | Primary: Resource Literacy | Supporting: C Programming
Depends on: 17E, 14B

21B Diagnose a font-selection boundary bug
Duration: 45m | Primary: Debugging and Testing | Supporting: C Programming
Depends on: 17E, 6A, 15B

22C Develop an original Pebble app concept
Duration: 60m | Primary: Idea Generation | Supporting: Resource Literacy
Depends on: 16B, 20C

23C Design a state machine for an app
Duration: 60m | Primary: Algorithmic Thinking | Supporting: UI Design
Depends on: 22C, 15B, 19G

24H Event-Driven Integration and Application Lifecycle
Duration: 150m | Primary: C Programming | Supporting: Development Tools and Workflow
Depends on: 19G, 23C

25C Design a multi-mode Pebble interface
Duration: 90m | Primary: UI Design | Supporting: Idea Generation
Depends on: 13B, 22C, 23C, 24H

26B Menus and scrolling
Duration: 75m | Primary: UI Design | Supporting: Algorithmic Thinking
Depends on: 17E, 24H

27C Button interactions
Duration: 60m | Primary: Algorithmic Thinking | Supporting: C Programming
Depends on: 12A, 24H

28D Custom drawing
Duration: 75m | Primary: Idea Generation | Supporting: UI Design
Depends on: 26B, 10D

29E Graphics paths
Duration: 75m | Primary: Algorithmic Thinking | Supporting: UI Design
Depends on: 28D, 18F

30G Accelerometer
Duration: 90m | Primary: C Programming | Supporting: Resource Literacy
Depends on: 24H, 20C

Phase IV - Graphics, hardware, validation and integration

Readiness checkpoint: Verify the three recurring projects, record limitations and diagnose integration faults.

31C Integrate font browsing with persistent selection
Duration: 150m | Primary: Development Tools and Workflow | Supporting: Debugging and Testing
Depends on: 25C, 24H, 14B, 7B

32C Diagnose a persistent-state integration bug
Duration: 90m | Primary: Debugging and Testing | Supporting: Development Tools and Workflow
Depends on: 24H, 21B, 31C

33F Bitmap manipulation
Duration: 90m | Primary: C Programming | Supporting: Debugging and Testing
Depends on: 29E, 18F

34H Magnetometer / compass
Duration: 90m | Primary: Resource Literacy | Supporting: C Programming
Depends on: 30G

35I Ambient-light limits and backlight
Duration: 45m | Primary: Resource Literacy | Supporting: UI Design
Depends on: 30G, 20C

36J Vibration
Duration: 45m | Primary: Debugging and Testing | Supporting: UI Design
Depends on: 30G, 24H

37K App timers
Duration: 75m | Primary: Debugging and Testing | Supporting: Algorithmic Thinking
Depends on: 27C, 24H

38L Resource files
Duration: 65m | Primary: Idea Generation | Supporting: Resource Literacy
Depends on: 33F, 20C

39M Persistent storage
Duration: 75m | Primary: Development Tools and Workflow | Supporting: Debugging and Testing
Depends on: 37K, 24H

40N Phone communication
Duration: 120m | Primary: Development Tools and Workflow | Supporting: C Programming
Depends on: 38L, 24H, 7B

Phase V - Independent application capstone

Readiness checkpoint: Build and defend an original working app using all seven competencies.

41A Build and demonstrate an original Emery app
Duration: 240m | Primary: Integrated capstone (all seven competencies) | Supporting: All seven competencies
Depends on: 22C, 25C, 32C, 36J, 39M, 40N

Three evolving project strands

Clock Workshop
Continuously revised lesson checkpoints: 2A -> 12A -> 27C -> 37K -> 39M | Learning trajectory: Time-only watchface reference -> two-window watchapp -> button behavior -> AppTimer -> persisted setting and recovery

Field Instrument
Continuously revised lesson checkpoints: 30G -> 34H -> 35I -> 36J | Learning trajectory: Accelerometer -> magnetic heading and calibration -> light API limitations -> intentional vibration and negative tests

Pocket Field Guide
Continuously revised lesson checkpoints: 26B -> 28D -> 29E -> 33F -> 38L -> 40N | Learning trajectory: Menu/scrolling -> custom drawing -> GPath -> writable bitmap -> raw resource -> phone messaging

Each new lesson starts from a completed prior-stage reference checkpoint, not a solution to its current exercise. Changes completed in a previous lesson should remain present in later examples. The 2A time-only watchface remains a watchface; the new interactive projects are watchapps. No dependency is imposed between unrelated project strands unless shared prerequisites truly require it. See the API coverage matrix.

Reading and exercises

C/SDK prerequisite policy: The 3A-10D framework-scaffolding guide explains graphics/timer callbacks already given to novices. These are not hidden prerequisites or additional tasks; 12A/27C/28D/37K assess those APIs later. Project continuity: A/B/C checkpoint matrix identifies behavior each next-stage starter must preserve.

3A-10D: Read the local chapters adapting Learning C with Pebble (Jipping, 2016), including the 11 distinct local Chapter 3-6 starter projects. 17E-24H: Read the original C-focused course notes on arrays, pointers, structs and event-driven lifecycles. 12A-40N: Consult each lesson's specific C API reference before editing its checkpoint. Keep coding exercises distinct from the written ideation, UI, tracing, resource-research and debugging exercises; none of these are optional simply because they produce no executable code.

Optional debugging challenges: 27C/debug (button navigation), 35I/debug (bad Light API call), 38L/debug (resource extraction); the original compulsory 6A/21B/32C remain. Optional challenges do not increase the required activity count or estimated total.

Independent demonstration - 41A

The capstone implements the original concept developed in 22C and UI groundwork in 25C. You must deliver a working, original Emery app, integrating at least two cross-strand capabilities (or an equivalent justified integration), and defend implementation decisions with source references, normal/failure tests, Git history and a demo. Score each of the seven competencies 0-3; every competency must reach 2+. Detailed capstone directions.

Course management and validation status

Use python3 taskwarrior/import_course.py to synchronize activities, prerequisites and learning objectives, preserving existing completed tasks and personal annotations. To inspect: task project:pebble.course list, task project:pebble.course unblocked, and task <ID> info. Always work in a personal branch/copy, with a known-good baseline.

Source/status transparency: All local lesson/project paths and Emery target manifests should be checked, but C code and phone integrations have not been fully compiled or physically validated in this editing environment. Do not treat a passing manifest audit as a passing SDK build. Maintain separate statuses: source supplied, build passed, emulator passed, real-device passed. Hardware-only validation can remain pending while continuing other independently unlocked tasks.

References

Jipping, M. (2016). Learning C with Pebble. GitBook. Local C course reading index.

Pebble Developers. (n.d.). C SDK documentation. https://developer.repebble.com/docs/c/

Reference links

Lesson directory and all 41 updated numbers : https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons

Legacy-to-sequential ID map : https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/taskwarrior/legacy-id-map.json

Competency and assessment rubric : https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/curriculum/course-design/learning-outcomes-and-assessment-.md

Official Pebble SDK reference : https://developer.repebble.com/docs/c/